Formal Security Analysis of the Post Quantum Shell Protocol

John G. Underhill

Quantum Resistant Cryptographic Solutions Corporation

June 2026

Abstract

The Post Quantum Shell (PQS) protocol is an application-layer remote shell, command, and file-transfer protocol carried inside the encrypted QSMS Simplex transport. QSMS establishes a server-authenticated post-quantum encrypted channel using a pinned server signature verification key, an ephemeral post-quantum key encapsulation exchange, SHA3 transcript binding, cSHAKE key derivation, and RCS authenticated encryption with the serialized QSMS packet header as associated data. PQS then performs application-layer login, command policy enforcement, sandboxed process execution, known-host continuity, and confined file transfer within the encrypted QSMS channel.

This paper gives an implementation-correspondent formal analysis of PQS. The analysis distinguishes QSMS transport security from PQS application-layer authorization, excludes unsupported claims of SSH wire compatibility, mutual cryptographic client authentication, post-compromise recovery, or ratchet-based recovery, and states the assumptions required for the implemented construction. The paper defines a server-authenticated Simplex AKE model, an authenticated-channel model for QSMS encrypted packets, and application-layer safety properties for login, command authorization, sandboxing, and file transfer.

Under the stated assumptions, the QSMS transport provides server authentication, session-key indistinguishability, header-bound ciphertext integrity, replay resistance subject to sequence and timestamp validation, and forward secrecy against later compromise of the long-term signing key. PQS application-layer security depends additionally on the secrecy of user passphrases, SCB verifier hardening parameters, policy correctness, sandbox configuration, file-system confinement, endpoint integrity, and administrative key distribution. These dependencies and limitations are made explicit.

Contents

1 Introduction

1.1 Scope

The analysis in this paper covers the implemented PQS system as a composition of two layers. The first layer is QSMS Simplex, which provides the cryptographic transport. The second layer is PQS, which provides remote shell, command, login, policy, sandbox, known-host, and file-transfer semantics inside the encrypted QSMS channel.

The paper treats the QSMS source code and the reference PQS specification as the authoritative description of implemented behavior. It does not analyze an idealized SSH-compatible protocol. PQS is not wire-compatible with SSH and does not implement the SSH transport, SSH user-authentication, or SSH connection protocols defined in RFC 4251, RFC 4252, RFC 4253, and RFC 4254. PQS is instead a separate post-quantum remote-administration protocol with a narrower and stricter application model.

1.2 Protocol Profile

The analyzed PQS profile uses QSMS rather than a native PQS packet layer for cryptographic transport. The QSMS serialized header order is flag, message length, sequence, and UTC timestamp. The QSMS exchange response is an RCS-authenticated confirmation message carrying the transcript confirmation value, not a bare success flag. The implemented transport uses RCS for authenticated encryption; AES-GCM is not part of the analyzed QSMS data path. The PQS application path does not claim post-compromise security through an active ratchet and does not authenticate the client cryptographically during the QSMS key exchange. The QSMS key-exchange model used here reflects the implementation path in which cSHAKE-256 is keyed by the KEM shared secret and customized by the finalized transcript hash; no additional KDF domain-label change is assumed.

1.3 Contributions

This work provides the following contributions.

1. A precise formal specification of the implemented QSMS Simplex handshake, including transcript updates and key confirmation.
2. A distinction between QSMS transport properties and PQS application-layer properties.
3. Security definitions for one-way server authentication, session-key indistinguishability, forward secrecy against later signing-key compromise, channel integrity, replay resistance, and downgrade resistance.
4. Application-layer analysis of SCB-based login, timing-neutral verification, command policy, sandboxed process execution, known-host pinning, and confined file transfer.
5. Explicit limitations covering endpoint compromise, password guessing, clock synchronization, denial of service, configured sandbox strength, and the absence of SSH compatibility or ratchet-based post-compromise recovery.

2 Related Work and Standards Context

Post-quantum KEM and signature standards have now been finalized for primary NIST migration paths. FIPS 203 specifies ML-KEM, FIPS 204 specifies ML-DSA, and FIPS 205 spec-

ifies SLH-DSA. The Keccak family is standardized through FIPS 202 for SHA3 and SHAKE, and SP 800-185 defines cSHAKE and KMAC as SHA3-derived functions. PQS and QSMS are analyzed here as a concrete protocol composition using these families or QRCS-configured alternatives.

The analysis also uses the conventional authenticated-key-exchange and authenticated-encryption literature. The QSMS transport is closest to a unilateral, server-authenticated AKE. It differs from mutually authenticated protocols because the server does not authenticate a cryptographic client identity during key exchange. This is a design boundary rather than a proof gap. The PQS application performs user authentication only after the transport channel is established.

Quantum model. Security claims involving post-quantum primitives are stated in the standard Q1 model: adversaries may perform quantum computation against public data and recorded transcripts, but protocol queries, signing queries, decapsulation queries, and application oracles are classical. The analysis does not claim Q2 security against superposition access to protocol oracles. A Q2 statement would require primitive assumptions and protocol definitions that explicitly support quantum oracle access; such assumptions are outside the scope of this paper.

SSH provides the closest operational comparison because it is the dominant remote-administration protocol family. The SSH architecture separates transport, user authentication, and connection protocols, and the SSH connection protocol multiplexes interactive sessions, command execution, forwarding, and X11 channels. PQS does not implement these wire protocols. Its narrower design permits a simpler post-quantum transport and a policy-centered application layer, but it also means that no result in this paper should be read as an SSH compatibility or SSH conformance result.

3 Implementation-Correspondence Summary

3.1 Transport and Application Boundary

QSMS owns the cryptographic handshake, packet header, sequence validation, freshness validation, RCS initialization, encryption, authenticated decryption, and transport error handling. PQS receives decrypted application messages from QSMS and applies application semantics. The first plaintext byte of a PQS application message is a PQS message type, such as login request, command request, file operation, status, error, or disconnect.

This separation is important for formal modeling. QSMS proves channel establishment and channel authenticity. PQS proves only the behavior of messages delivered after QSMS has established the encrypted channel. PQS application login is not part of the QSMS AKE proof. Conversely, QSMS key exchange does not depend on PQS usernames, passphrases, shell profiles, or file-transfer paths.

3.2 Implemented Primitive Profile

The default implemented QSMS profile uses a post-quantum signature scheme for server authentication, a post-quantum KEM for encapsulation, SHA3-256 for transcript hashes, cSHAKE-256 for key derivation, and RCS for authenticated encryption. The configuration mechanism allows alternative compile-time suites, including Dilithium/Kyber, Dilithium/McEliece, and SPHINCS+/McEliece profiles. The default supplied profile is Dilithium/Kyber with SHA3 and RCS.

For formal notation, this paper uses ML-DSA and ML-KEM where the default profile maps to the corresponding standardized post-quantum families. Where the implementation is configured for McEliece or SPHINCS+, the same theorem structure applies after replacing the KEM or signature assumptions by the corresponding IND-CCA or EUF-CMA assumptions. The names Kyber and Dilithium as supplied by the QSC cryptographic library denote the NIST-competition primitives that were standardized as ML-KEM (FIPS 203) and ML-DSA (FIPS 204); the standardization process altered domain separation and key-derivation details, so the ML-KEM/ML-DSA labels are used here at the level of the underlying IND-CCA and EUF-CMA hardness assumptions rather than as a claim of byte-for-byte conformance with FIPS 203/204. A deployment requiring FIPS conformance must confirm that the configured QSC backend implements the standardized parameter sets.

Transport parameter source. The transport-layer values in the table below (header layout and field widths, time threshold, RCS key/nonce/tag sizes, and the cSHAKE key-schedule block) are defined and serialized by the QSMS transport implementation. PQS consumes the established QSMS channel and does not re-implement the byte order, sequence validation, timestamp validation, associated-data binding, or RCS packet processing rules.

3.3 Implementation Parameters Used in the Analysis

Parameter	Implemented value	Security role
QSMS header size	21 bytes	Authenticated transport metadata.
Message length field	32-bit little-endian	Bounds body allocation and parse size before allocation or decryption.
Sequence field	64-bit little-endian	Strict replay and order check. QSMS uses exact sequence equality, not a replay window.
Timestamp field	64-bit little-endian UTC	Freshness check for handshake and encrypted packets.
QSMS packet time threshold	60 seconds	Two-sided clock tolerance for valid-time rejection.
KDF output block	One cSHAKE-256 rate block	Directional RCS keys and nonces, plus residual state material from which the implementation stores a ratchet seed. The reference PQS application path does not claim ratchet-based post-compromise recovery.
RCS key size	32 bytes per direction	Transport confidentiality and integrity for each direction.
RCS nonce size	32 bytes per direction	Directional RCS initialization.
RCS tag size	32 bytes	Packet authentication and ciphertext integrity.
PQS login verifier	SCB over domain, user, passphrase, and salt	Application-layer user authentication after QSMS establishment.

These parameters are not independently negotiated at runtime in the default QSMS path. They are selected by implementation configuration and are bound into the session through the configuration string and transcript hash.

4 Notation and Definitions

Let C denote the client and S denote the server. Byte-string concatenation is written $x \parallel y$. The empty string is written ϵ. Sampling from a probabilistic algorithm A is written $x \leftarrow A(\cdot)$. The output length of SHA3-256 is 32 bytes. The QSMS key identifier is 16 bytes. The QSMS header is 21 bytes.

The server long-term signature key pair is $(\mathsf{ssk}, \mathsf{pvk})$. The client holds a pinned verification record

$$(\mathsf{cfg}, \mathsf{kid}, \mathsf{pvk}, \mathsf{expiration}).$$

The field cfg is the QSMS configuration string, kid is the key identifier, pvk is the server verification key, and expiration is the UTC expiration time of the verification-key record.

The ephemeral KEM public and private keys are $(\mathsf{pk}, \mathsf{sk})$. The KEM ciphertext is cpt, and the KEM shared secret is sec. The evolving QSMS transcript hash is written sch. For clarity, the intermediate transcript hashes are written sch_0, sch_1, and sch_2.

5 QSMS Packet and State Model

5.1 Header Serialization

A QSMS network packet consists of a fixed header and a variable message body. The header fields are

$$\mathsf{hdr} = (\mathsf{flag}, \mathsf{mlen}, \mathsf{seq}, \mathsf{utc}).$$

The canonical serialized header is

$$\mathsf{encode}(\mathsf{hdr}) = \mathsf{flag} \parallel \mathsf{LE32}(\mathsf{mlen}) \parallel \mathsf{LE64}(\mathsf{seq}) \parallel \mathsf{LE64}(\mathsf{utc}).$$

The serialized header length is $1 + 4 + 8 + 8 = 21$ bytes. This exact serialized byte string is used in two distinct places: it is hashed into the signed Connect Response public-key binding, and it is supplied as RCS associated data for encrypted QSMS packets.

5.2 Relevant QSMS Flags

The QSMS flags relevant to the reference PQS application path are:

Flag	Value	Role
none	0x00	Initial state.
connect request	0x01	Client-to-server KEX request.
connect response	0x02	Server-to-client signed ephemeral KEM key.
connection terminate	0x03	Transport termination condition.
encrypted message	0x04	Established encrypted channel packet.
exchange request	0x07	Client-to-server KEM ciphertext.
exchange response	0x08	Server-to-client RCS-authenticated key confirmation.
session established	0x0D	Established local transport state.
session establish verify	0x0E	Internal verification state.
error condition	0x14	Transport error packet.

Other flags are implemented for additional QSMS transport control paths, including exstart, establish, remote-status, ratchet, transfer, and error handling. They are not necessary to prove

the reference PQS remote-shell application path. The flag values listed above are QSMS transport flags; PQS application-layer message types (Section 8) are distinct plaintext values carried inside encrypted QSMS messages.

5.3 Header Validation

For handshake packets, QSMS validates the expected input state, expected output flag, exact sequence number, exact message length, and timestamp freshness. For encrypted data packets, QSMS requires the established session state, strict sequence advancement, a fresh timestamp, a message length greater than the RCS tag length, and successful RCS authentication. The timestamp predicate accepts a packet only when the sender timestamp is within the configured two-sided QSMS packet time threshold relative to local UTC time.

The implementation uses strict sequence equality rather than a replay window. Therefore the formal acceptance predicate for data packets is

$$\mathsf{hdr.seq} = \mathsf{rxseq} + 1.$$

Upon successful acceptance, the receiver increments rxseq. On authentication failure, the receive sequence is not advanced.

6 QSMS Simplex Handshake Specification

6.1 Connect Request

The client selects a pinned verification record and rejects it if its expiration time has passed. It constructs

$$m_1 = \mathsf{kid} \parallel \mathsf{cfg}$$

and sends a Connect Request packet

$$C \rightarrow S : \mathsf{CRQ} = \mathsf{hdr}_1 \parallel m_1,$$

where hdr_1 has flag Connect Request, sequence txseq, message length $|m_1|$, and the current UTC timestamp.

The client also computes the initial session cookie

$$\mathsf{sch}_0 = H(\mathsf{cfg} \parallel \mathsf{kid} \parallel \mathsf{pvk}).$$

The server validates hdr_1, parses m_1, looks up kid, checks cfg against its configured protocol string, checks key expiration, and computes the same sch_0 from its stored key record.

6.2 Connect Response

The server generates a fresh ephemeral KEM key pair

$$(\mathsf{pk}, \mathsf{sk}) \leftarrow \mathsf{KEM.KeyGen}().$$

It prepares a Connect Response header hdr_2 with message length equal to the signature output plus the ephemeral KEM public key. It then computes

$$\mathsf{phash} = H(\mathsf{encode}(\mathsf{hdr}_2) \parallel \mathsf{pk})$$

and signs this hash:

$$\sigma \leftarrow \mathsf{SIG.Sign}(\mathsf{ssk}, \mathsf{phash}).$$

The implementation stores the signature material in the response body ahead of pk and sends

$$S \rightarrow C : \mathsf{CRP} = \mathsf{hdr}_2 \parallel \sigma \parallel \mathsf{pk}.$$

The server updates its transcript hash as

$$\mathsf{sch}_1 = H(\mathsf{sch}_0 \parallel \mathsf{phash}).$$

The client validates hdr_2, verifies the signature under the pinned pvk, recomputes phash as $H(\mathsf{encode}(\mathsf{hdr}_2) \parallel \mathsf{pk})$, and accepts the Connect Response only if the verified signed hash equals the recomputed hash. It then updates its transcript hash to sch_1.

6.3 Exchange Request

The client encapsulates against the server ephemeral KEM public key:

$$(\mathsf{cpt}, \mathsf{sec}) \leftarrow \mathsf{KEM.Encaps}(\mathsf{pk}).$$

It updates the transcript hash as

$$\mathsf{sch}_2 = H(\mathsf{sch}_1 \parallel \mathsf{cpt})$$

and sends

$$C \rightarrow S : \mathsf{XRQ} = \mathsf{hdr}_3 \parallel \mathsf{cpt},$$

where hdr_3 has flag Exchange Request and message length $|\mathsf{cpt}|$.

The server validates hdr_3 and decapsulates

$$\mathsf{sec} \leftarrow \mathsf{KEM.Decaps}(\mathsf{sk}, \mathsf{cpt}).$$

It updates sch_2 using the received ciphertext. If decapsulation fails, the server emits an error and clears provisional state.

6.4 Key Schedule

Both endpoints invoke cSHAKE-256 using sec as the input keying material and sch_2 as the customization string. In implementation notation this corresponds to

$$\mathsf{prnd} \leftarrow \mathsf{cSHAKE}_{256}(\mathsf{key} = \mathsf{sec}, \mathsf{custom} = \mathsf{sch}_2).$$

One Keccak-rate block is squeezed and parsed as

$$\mathsf{prnd} = k_1 \parallel n_1 \parallel k_2 \parallel n_2 \parallel r,$$

where k_1, k_2 are 32-byte RCS keys, n_1, n_2 are 32-byte RCS nonces, and r is residual material. The residual Keccak state is permuted and a portion is stored as a ratchet seed, but the reference PQS application path does not perform a completed ratchet and this paper does not claim post-compromise recovery from that seed.

Directional assignment is

$$C_{\mathsf{tx}} = (k_1, n_1), \quad C_{\mathsf{rx}} = (k_2, n_2),$$
$$S_{\mathsf{rx}} = (k_1, n_1), \quad S_{\mathsf{tx}} = (k_2, n_2).$$

The key schedule output, KEM shared secret, and temporary key-parameter structures are erased after cipher initialization.

6.5 Exchange Response and Key Confirmation

The server constructs an Exchange Response packet with message length

$$|\mathsf{sch}_2| + |\mathsf{tag}|,$$

sets the serialized header as RCS associated data, and encrypts the transcript confirmation value sch_2 using its transmit RCS state:

$$\mathsf{ct}_4 \parallel \mathsf{tag}_4 \leftarrow \mathsf{RCS.Enc}(S_{\mathsf{tx}}, \mathsf{encode}(\mathsf{hdr}_4), \mathsf{sch}_2).$$

It sends

$$S \rightarrow C : \mathsf{XRP} = \mathsf{hdr}_4 \parallel \mathsf{ct}_4 \parallel \mathsf{tag}_4.$$

The client validates the Exchange Response header, sets the serialized header as RCS associated data, decrypts using C_{rx}, and accepts only if the decrypted value equals its local sch_2. This is the explicit QSMS key-confirmation step. It is not a separate fifth network message.

7 QSMS Encrypted Data Channel

7.1 Encryption

After QSMS enters the established state, each PQS application payload m is transmitted in a QSMS encrypted message packet. The sender increments its transmit sequence counter and builds a header hdr with flag Encrypted Message and message length $|m| + 32$, since the implemented RCS tag is 32 bytes. It serializes the header, supplies it as associated data, and computes

$$\mathsf{ct} \parallel \mathsf{tag} \leftarrow \mathsf{RCS.Enc}(k, n, \mathsf{encode}(\mathsf{hdr}), m).$$

The transmitted packet is hdr ‖ ct ‖ tag.

7.2 Decryption

The receiver accepts an encrypted message only if the sequence equals rxseq + 1, the connection is established, the timestamp is valid, and the body length is greater than the RCS tag length. It supplies the serialized header as associated data and verifies the RCS tag before releasing plaintext. If tag verification succeeds, the receiver increments rxseq and returns the plaintext to the PQS application callback. If tag verification fails, no plaintext is released and rxseq is not advanced.

7.3 Replay and Header Binding

The RCS tag authenticates the serialized header and therefore covers the flag, message length, sequence, and UTC timestamp. A replayed packet carries the same authenticated sequence number and is rejected because the receive counter has advanced. An attacker cannot alter the sequence or timestamp to pass header validation without producing a new valid RCS tag.

8 PQS Application-Layer Specification

8.1 Application State

PQS application state is separate from QSMS transport state. The reference application states are none (0x00), connected (0x01), login required (0x02), authenticated (0x03), command ac-

tive (0x04), and closing (0x05). The reference console server enforces a single active application session and refuses concurrent client connections; it accepts command, file-operation, and typed-administrative messages only after the application state has reached authenticated.

8.2 Login and SCB Verifier

The login request is a PQS application message inside the encrypted QSMS channel. It carries bounded fixed-size username and passphrase fields. A server user record stores username, enabled state, privilege, shell profile, failure counter, salt, and verifier. The passphrase itself is not stored.

Let dom denote the PQS verifier domain string, u the username, p the passphrase, salt the per-user salt, and SCB the configured password-hardening function. The implemented verifier profile is

$$\mathsf{seed} = H(\mathsf{dom} \parallel u \parallel p),$$

$$\mathsf{verifier} = \mathsf{SCB}(\mathsf{seed}, \mathsf{salt}, \mathsf{cpu\_cost}, \mathsf{memory\_cost}).$$

The comparison uses the QSC constant-time verification routine. The timing-neutral login path evaluates a dummy verifier path for missing, disabled, or locked records before returning failure. A persistent failed-attempt counter is enforced, and a user record that reaches the configured threshold is disabled until administrative action changes the record state.

The cost parameters cpu_cost and memory_cost are exposed as the build-time macros PQS_CRYPTO_PHASH_CPU_COST and PQS_CRYPTO_PHASH_MEMORY_COST in pqs.h. The default values shipped in the reference source are $\mathsf{cpu\_cost} = 4$ and $\mathsf{memory\_cost} = 1$ MiB. These defaults are modest, and the offline-guessing analysis of Section 16.3 should be read with the concrete deployed parameters in mind: the hardness of the verifier against offline attack is only as strong as the configured cost, and operators are expected to raise these parameters for production use. The verifier domain string is bound into the seed as the fixed label PQS-USER-SCB-VERIFIER-V2; changing it or the cost parameters changes the verifier derivation profile and requires a database migration or verifier reset.

8.3 Command Authorization and Process Confinement

A command request (application message type 0x20) is processed only after authentication. The server resolves the user, privilege, shell profile, command policy, and sandbox profile. The policy modes are no-shell, restricted, forced, and raw-shell. Restricted mode authorizes commands by the leading command verb against a per-policy allow list. Forced mode substitutes a configured command. Raw-shell mode allows shell execution subject to a per-policy deny list and the command-safety predicate. A leading-verb deny-list check and the command-safety predicate are applied in *all* non-deny modes, including the forced and raw-shell paths.

The command-safety predicate is implemented as a *safe-character allow list* rather than a denied-metacharacter list: a command line is accepted only if every byte is an ASCII letter, digit, space, tab, or one of _ - . / \ : = + ,. Any other byte, including the shell-control metacharacters that enable command chaining, substitution, or redirection (for example ; | & $ ` > < ( ) { } ' " * ? and newline), causes rejection before the process module is invoked. This prevents an allowed verb from carrying an additional shell expression on the same command line. The predicate constrains the byte alphabet only; it does not assert semantic safety of the allowed program or of arguments expressible within the safe alphabet.

A separate typed administrative channel exists (client administrative request `0x23`, server responses `0x24`/`0x25`). Administrative requests are accepted only when the session is authenticated, the active user privilege is administrative, and the requested administrative verb is authorized by the policy store; otherwise the request is denied and audit-logged. This channel carries structured, enumerated administrative verbs rather than free-form shell text, and is therefore analyzed as a privilege-gated, policy-authorized command path distinct from the shell command path above.

Process creation is delegated to the PQS process module. That module controls descriptor or handle inheritance, redirects child standard input where appropriate, applies timeout limits, clears or constructs the environment according to the sandbox profile, applies POSIX privilege-drop and no-new-privileges where configured and available, and uses Windows restricted-token or job-object limits where available. The default policy assignment grants restricted mode to guest and user privilege classes and raw-shell mode to the administrative class; the formal command-execution property is therefore conditional on correct policy and sandbox configuration and on operating-system enforcement.

8.4 File Transfer

PQS file transfer is policy-gated and authenticated. The application message types include file get (`0x40`), put start (`0x41`), put data (`0x42`), put final (`0x43`), list (`0x44`), mkdir (`0x45`), remove (`0x46`), recursive get (`0x4A`), and the server-originated data, finalization, status, and recursive directory/file markers.

The file-transfer module validates relative paths, rejects empty paths, absolute paths, drive-qualified paths, traversal components (a `..` path element), and oversized paths, and constructs per-user transfer roots. The implementation provides two distinct enforcement mechanisms with different strength, and this distinction is material to the confinement claim of Section 15.3:

- *Read and write of regular files* (get and put) use a confined open helper. On POSIX this helper opens the transfer root directory and then resolves each path component with `openat` under `O_NOFOLLOW` (and `O_DIRECTORY` for non-final components), so that symlink traversal is rejected *during* resolution rather than by a prior string check. On Windows the helper performs canonical root-prefix validation and rejects reparse-point targets before opening.

- *Directory creation, file removal, and atomic publish/rename* do not use descriptor-relative resolution. They construct a local path string, validate it with a canonicalization-plus-prefix check (`realpath` on POSIX, canonical prefix on Windows) together with a separate `lstat`-based symbolic-link test, and then perform the `mkdir`, delete, or `rename` on that path. This is a check-then-use sequence: the validation and the mutating operation are not atomic with respect to concurrent changes to intermediate directory components.

Recursive traversal is centralized in the PQS transfer walker, is bounded by a fixed recursion limit (`PQS_XFER_RECURSION_MAX` $= 64$), and skips symbolic links or reparse points.

For a transferred file byte string F, the finalization metadata carries $|F|$ and

$$h_F = \mathsf{SHA3\text{-}256}(F).$$

The receiver accepts completion only if the observed byte count and computed hash match the metadata.

9 Security Model

9.1 Adversary

The adversary controls the network: it may read, drop, delay, replay, reorder, inject, and modify packets. It may start concurrent client or server sessions. It may compromise long-term server signing keys after a target session, unless excluded by a freshness condition. It may compromise application passwords only through online or offline guessing; such guessing is analyzed separately and depends on SCB cost, salt uniqueness, password entropy, and account-lockout policy.

The model assumes correct implementations of ML-KEM, ML-DSA or the configured alternative signature, SHA3-256, cSHAKE-256, RCS, constant-time comparison primitives, secure random generation, and secure erasure. The model excludes physical side channels, malware on endpoints, malicious administrators, compromised operating systems, and failures of file-system or sandbox primitives.

9.2 Freshness

A QSMS client session is fresh if the session has accepted through a valid Exchange Response, neither endpoint session state was compromised before acceptance, the long-term signing key was not compromised before the Connect Response was created, and the adversary has not obtained the KEM shared secret or RCS keys for the target session.

Later compromise of the server long-term signing key is permitted for forward-secrecy analysis. Later compromise of a session RCS key trivially exposes that session from the compromise point and does not provide post-compromise recovery in the reference PQS application path.

9.3 Security Goals

The intended goals are:

1. server authentication in QSMS;
2. session-key indistinguishability for fresh QSMS sessions;
3. forward secrecy with respect to later compromise of the long-term signing key;
4. RCS confidentiality and ciphertext integrity for established packets;
5. replay rejection through strict sequence validation and timestamp validation;
6. configuration binding through the transcript hash;
7. application login resistance to offline verifier cracking proportional to passphrase entropy and SCB cost;
8. command authorization and file confinement under correct policy and sandbox configuration.

10 Formal Experiments

10.1 Server-Authentication Experiment

The challenger generates a server signing key pair and gives the verification key to the client as a pinned record. The adversary controls all network messages. The adversary wins if a fresh

client accepts a QSMS session using a Connect Response for which no honest server instance produced the signed value

$$H(\mathsf{encode}(\mathsf{hdr}_2) \parallel \mathsf{pk}).$$

The experiment excludes trivial wins after pre-acceptance compromise of the long-term signing key.

10.2 Session-Key Experiment

The challenger runs the QSMS state machine. For a fresh accepted client session, the adversary may issue a test query. The challenger returns either the real directional RCS key and nonce vector or a uniform vector of the same length. The adversary wins if it identifies which value was returned with probability exceeding one half. The freshness predicate forbids prior disclosure of the target KEM secret, target session state, or target RCS keys.

10.3 Packet-Integrity Experiment

After a session is established, the adversary may obtain encryptions of chosen application messages in each direction and may submit ciphertexts for decryption, subject to standard no-trivial-replay restrictions. It wins if it produces a new packet that passes QSMS header validation and RCS authentication but was not produced by the honest sender for that direction.

10.4 Replay Experiment

The adversary observes an accepted packet and later resubmits it, or resubmits it with modified metadata. It wins if the packet is accepted a second time in the same session direction. This game is won only if strict sequence validation fails, timestamp validation fails, or the adversary forges a valid RCS tag for modified associated data.

10.5 Application-Safety Experiment

For the PQS application layer, define an authenticated user session as one in which QSMS is established and the server has returned login success for a valid enabled user record. The adversary wins the command-safety game if an unauthenticated session executes a command, if a restricted-policy session executes a command not authorized by policy except through an administrator-configured forced or raw-shell policy, or if a non-administrative or unauthenticated session causes a typed administrative verb to execute. The adversary wins the file-confinement game if a regular-file read or write reaches a path outside the authenticated user's transfer root while satisfying the implemented path-validation and confined-open checks. (Directory-mutating operations are analyzed separately under the TOCTOU caveat of Section 15.3, which adds a no-concurrent-mutation assumption.) These application games are conditional on a non-compromised server endpoint and correct operating-system enforcement.

11 Transport Security Theorems

11.1 Server Authentication

Theorem 1 (Server authentication)**.** *Let $\mathcal{A}$ be an adversary that causes a fresh client to accept a QSMS session with a Connect Response not produced by the holder of* ssk *for the pinned* pvk.

Then there exists an adversary $\mathcal{B}$ that forges a signature under the configured server signature scheme with comparable advantage, except for the probability of a SHA3 collision in the signed hash input.

Proof. The client accepts the Connect Response only after verifying a signature over

$$\mathsf{phash} = H(\mathsf{encode}(\mathsf{hdr}_2) \parallel \mathsf{pk}).$$

If $\mathcal{A}$ changes hdr_2 or pk while preserving acceptance, then either the recomputed phash collides with the signed value or $\mathcal{A}$ has produced a valid signature on a new hash value under pvk. The first event is bounded by the collision resistance of SHA3-256; the second is an EUF-CMA forgery for the signature scheme. Therefore an unauthenticated acceptance implies one of these events. □

11.2 Session-Key Indistinguishability

Theorem 2 (Session-key indistinguishability)**.** *For any adversary $\mathcal{A}$ attacking a fresh QSMS session, there exist adversaries $\mathcal{B}_1, \mathcal{B}_2, \mathcal{B}_3$ such that*

$$\mathsf{Adv}^{\mathsf{sk}}_{\mathsf{QSMS}}(\mathcal{A}) \le \mathsf{Adv}^{\mathsf{IND\text{-}CCA}}_{\mathsf{KEM}}(\mathcal{B}_1) + \mathsf{Adv}^{\mathsf{EUF\text{-}CMA}}_{\mathsf{SIG}}(\mathcal{B}_2) + \mathsf{Adv}^{\mathsf{PRF}}_{\mathsf{cSHAKE}}(\mathcal{B}_3) + \varepsilon_H.$$

Here ε_H accounts for the collision probability of the transcript hashes.

Proof. The proof uses a standard hybrid argument. In the first hybrid, replace the KEM shared secret sec in the target session by a uniform string. Distinguishing this hybrid from the real execution gives an IND-CCA adversary against the KEM. In the second hybrid, exclude unauthenticated Connect Responses by the server-authentication theorem. In the third hybrid, replace the cSHAKE output keyed by sec and customized by sch_2 with uniform random material. Distinguishing this hybrid from the previous one gives a PRF distinguisher against cSHAKE. In the final hybrid, k_1, n_1, k_2, n_2 are uniformly random and independent of the adversary view, so the adversary's distinguishing advantage is zero. Summing the transition distances gives the stated bound. □

11.3 Forward Secrecy

Theorem 3 (Forward secrecy against later signing-key compromise)**.** *If a QSMS session is fresh through completion and the ephemeral KEM private key and KEM shared secret are erased after key derivation, later compromise of the server long-term signing key does not reveal the completed session RCS keys unless the KEM or cSHAKE assumptions fail.*

Proof. The long-term signing key authenticates the ephemeral KEM public key but is not an input to the cSHAKE key schedule. The derived RCS keys depend on sec and sch_2. After session completion, disclosure of ssk gives the adversary the ability to sign future Connect Responses but does not reveal the past KEM secret. By IND-CCA security of the KEM and PRF security of cSHAKE, the past RCS keys remain indistinguishable from random, assuming the endpoint erased the ephemeral KEM private key, KEM secret, and derived temporary material. □

11.4 Key Confirmation

Lemma 1 (QSMS key confirmation)**.** *A fresh client accepts the QSMS exchange response only if the server, or an adversary who knows the derived server-to-client RCS key, produced a valid RCS ciphertext of the transcript confirmation value* sch_2 *under associated data* $\mathsf{encode}(\mathsf{hdr}_4)$.

Proof. The client decrypts the Exchange Response using its receive RCS state and the serialized Exchange Response header as associated data. It then compares the recovered plaintext to its locally computed transcript confirmation value. RCS releases plaintext only after tag verification. Therefore acceptance requires knowledge of the correct RCS key or a successful RCS forgery. □

Remark 1. The confirmed plaintext is written here as sch_2 for uniformity with the key-schedule notation. The exact confirmation value is fixed by the QSMS serializer (the companion QSMS/QSMP material derives the confirmation from a hash of the finalized transcript rather than the raw transcript bytes); this does not affect the lemma, which depends only on the value being a deterministic function of the agreed transcript that both endpoints compute independently. A conforming implementation must use the same confirmation derivation on both sides.

12 Data-Channel Security

12.1 Confidentiality

For each established packet, the plaintext is encrypted under the directional RCS key. Given session-key indistinguishability and RCS confidentiality, application plaintexts are indistinguishable from random to a network adversary, except for metadata visible in the QSMS header and transport layer: timing, packet length, packet count, direction, connection endpoint, and coarse application behavior inferred from traffic shape.

12.2 Integrity and Non-Malleability

Theorem 4 (Packet integrity)**.** *An adversary that produces a new encrypted QSMS packet that passes header validation and RCS authentication yields an adversary against the INT-CTXT security of RCS, except for negligible header-collision probability.*

Proof. The receiver supplies $\mathsf{encode}(\mathsf{hdr})$ as associated data before RCS decryption. Any change to flag, message length, sequence, timestamp, ciphertext, or tag changes the verification input. If the modified packet is accepted, then the adversary has produced a valid RCS ciphertext/tag pair under associated data not previously generated by the encryption oracle, which is an INT-CTXT forgery. □

12.3 Replay Resistance

A previously accepted encrypted packet cannot be accepted again in the same direction because the receive sequence has advanced. A previously generated ciphertext also cannot be modified to carry a new sequence or timestamp because those fields are authenticated. Timestamp validation supplies an independent freshness check and rejects packets outside the configured clock window. Replay resistance therefore follows from strict sequence validation plus RCS associated-data binding.

12.4 Downgrade Resistance

The transcript hash binds the configuration string, key identifier, verification key, signed ephemeral public-key binding, and KEM ciphertext through

$$\mathsf{sch}_0 = H(\mathsf{cfg} \parallel \mathsf{kid} \parallel \mathsf{pvk}), \quad \mathsf{sch}_1 = H(\mathsf{sch}_0 \parallel \mathsf{phash}), \quad \mathsf{sch}_2 = H(\mathsf{sch}_1 \parallel \mathsf{cpt}).$$

An attacker who alters cfg or substitutes another verification key either fails explicit configuration and key checks or causes a different transcript hash and therefore different derived keys. The authenticated Exchange Response then fails. Because the protocol does not negotiate fallback algorithms at runtime in the reference implementation, the downgrade surface is limited to misconfigured or maliciously provisioned pinned keys and compile-time configuration choices.

13 Layer-Composition Result

Theorem 5 (QSMS-PQS composition)**.** *Assume the QSMS transport establishes a fresh authenticated channel with confidential and integrity-protected delivery, and assume the PQS server processes only plaintext delivered by QSMS after the established state. Then a network adversary that does not compromise an endpoint cannot cause PQS to accept an unauthenticated command, typed administrative request, or file operation except by either breaking QSMS channel integrity, forging a valid login, or exploiting an application authorization or sandbox defect.*

Proof. Before QSMS establishment, the PQS application callback receives no decrypted application messages. After QSMS establishment, every application message delivered to PQS has passed RCS authentication under associated data containing the QSMS header. Therefore a network adversary cannot inject or modify a PQS application message without an RCS forgery. The server-side PQS dispatcher denies the command, typed-administrative, and file-operation message types unless the application state is authenticated; the typed-administrative path additionally requires administrative privilege and policy authorization. Thus an unauthorized command, administrative request, or file operation requires either a forged encrypted message that bypasses QSMS, a successful login under the user verifier policy, or a defect or misconfiguration in the application authorization path. □

This composition theorem is intentionally narrow. It does not prove that every authorized command is safe, that every operating system sandbox is complete, or that all side channels are absent. It proves that the implemented layering gives the application a cryptographically authenticated input boundary before policy is evaluated.

14 Application-Layer Security Analysis

14.1 Login Security

The QSMS channel protects login messages in transit. Server-side login security then depends on passphrase entropy, salt uniqueness, SCB cost parameters, secure storage of the verifier database, and enforcement of login failure policy. Binding the verifier domain string and username into the seed prevents verifier equivalence across user names for the same passphrase. Timing-neutral dummy verification reduces gross username enumeration by making missing, disabled, and locked account paths perform comparable verifier work before returning failure.

This does not eliminate all possible side channels. Disk I/O, cache state, account-record location, logging, network scheduling, and administrative configuration may still produce measurable differences. The formal claim is therefore limited to reducing the primary branch-level timing oracle in the application verifier path.

14.2 Command Execution Security

The application command model provides authorization only after successful login. The command policy engine rejects shell-control metacharacters in user-supplied command lines before executing commands through a shell. This design rule addresses the direct policy-bypass class in which a permitted verb is followed by additional shell syntax.

Command safety nevertheless depends on the configured shell profile, allowed commands, forced command strings, environment policy, working directory, operating-system privilege boundaries, and sandbox enforcement. If an administrator enables raw-shell execution for a broad privilege class or configures a shell profile with excessive privilege, the protocol cannot prevent authorized but unsafe commands. The formal guarantee is therefore a policy-enforcement guarantee, not a guarantee that all administrator-approved commands are harmless.

14.3 Process Isolation

Descriptor and handle inheritance control prevents command children from unintentionally inheriting server sockets, key files, database descriptors, and other long-lived resources. POSIX confinement may include descriptor closure, controlled standard streams, environment clearing, no-new-privileges, chroot, and run-as user/group settings. Windows confinement may include explicit inherited-handle lists, restricted tokens, and job-object limits. These mechanisms are platform dependent. The analysis assumes the operating system enforces the requested restrictions correctly.

14.4 File-Transfer Security

The file-transfer subsystem is designed to preserve the transfer-root boundary. The key invariant is that an authenticated user's file operations resolve only to paths inside that user's configured transfer root. For regular-file read and write, POSIX descriptor-relative traversal with no-follow semantics is stronger than string-only path checks because intermediate symlink traversal is rejected during path resolution, and Windows canonical-path validation with reparse-point rejection provides the analogous boundary check within the platform file-system model.

Directory creation, file removal, and atomic publish/rename do not use the descriptor-relative open helper. They validate a constructed path string by canonicalization-plus-prefix comparison and a separate symbolic-link test, and then perform the mutating call on that path. Because validation and mutation are not atomic, these operations are subject to a time-of-check-to-time-of-use (TOCTOU) race if an adversary with concurrent write access to an intermediate directory can substitute a component (for example a symlink) between the check and the call. The confinement guarantee for these operations therefore additionally assumes the absence of concurrent adversarial mutation of the transfer-root subtree, beyond the operating-system assumptions that suffice for the descriptor-relative open path. A descriptor-relative mutation profile using `mkdirat`, `unlinkat`, and `renameat` under the same no-follow component walk used for file open would remove this additional assumption. The caveat does not affect the get/put confinement guarantee analyzed in Section 15.3.

The file hash $h_F = \mathsf{SHA3\text{-}256}(F)$ verifies accidental or adversarial mismatch at transfer completion. Since file data is already carried inside QSMS encrypted packets, the transfer hash is an application integrity check over the assembled file, not the primary network authentication mechanism. The primary network integrity mechanism remains RCS.

14.5 Known-Hosts and Pinning

The client public verification key is the trust anchor for QSMS server authentication. The known-hosts database maps a host identifier to the SHA3-256 fingerprint of the server public verification record. Strict host-key checking rejects unknown keys, while trust-on-first-use mode can record an unknown key after operator approval. A changed fingerprint is rejected. These mechanisms protect against unexpected server-key substitution only if the initial trust anchor or first-use acceptance is authentic.

15 Application-Layer Theorems

15.1 Login Acceptance

Lemma 2 (Login acceptance)**.** *In a non-compromised server, a PQS login succeeds only if the submitted passphrase, together with the submitted username and verifier domain, reproduces the stored SCB verifier for an enabled and unlocked user record.*

Proof. The login handler parses the fixed-size login payload inside the encrypted QSMS channel. It locates the user record and invokes the timing-neutral verifier path. For a usable record, the verifier path computes the seed from domain, username, and passphrase, applies SCB with the stored salt and configured cost parameters, and compares against the stored verifier using a constant-time comparison. Login success is returned only on a successful comparison and usable account state. Missing, disabled, or locked records execute a dummy verifier path and return failure. □

15.2 Restricted Command Policy

Lemma 3 (Restricted command policy)**.** *For a restricted-policy user, a user-supplied command line that reaches process execution has a leading command verb on the policy allow list and not on the deny list, and consists exclusively of bytes in the command-safety allow alphabet.*

Proof. The command handler requires the authenticated application state. It resolves the policy associated with the user's privilege class. The command-safety predicate accepts the command line only if every byte lies in the safe alphabet of Section 8.3; this rejects all shell-control metacharacters before the process module is invoked. The engine then extracts the leading verb, rejects it if present on the deny list, and in restricted mode requires it on the allow list. Therefore a command reaching execution under restricted policy has an allow-listed, non-deny-listed leading verb and contains no byte outside the safe alphabet. This does not assert semantic safety of the allowed program or of arguments expressible within the safe alphabet. □

15.3 Transfer-Root Confinement

Lemma 4 (Transfer-root confinement for confined open)**.** *Assume correct operating-system enforcement of descriptor-relative no-follow traversal on POSIX, and canonical root-prefix plus reparse-point checks on Windows. Then a regular-file read or write accepted by the confined open helper resolves to an object inside the authenticated user's transfer root.*

Proof. The relative path validator rejects absolute, drive-qualified, traversal, empty, and oversized paths. POSIX confined opening starts from a directory descriptor for the transfer root and resolves each component relative to the current descriptor, using `O_NOFOLLOW` to reject symlink

traversal during resolution. Windows confined opening canonicalizes the candidate path, checks that the canonical target remains under the canonical root prefix, and rejects reparse-point targets. Under the operating-system assumptions, an accepted handle therefore refers to an object inside the transfer root. □

Remark 2. The lemma is stated for the confined open helper, which serves the get and put paths. Directory creation, removal, and rename use a canonicalization-plus-prefix check followed by a non-atomic mutating call (Section 8.4); their confinement holds only under the additional assumption that no adversary concurrently mutates an intermediate directory component between the check and the call. The lemma does not cover those operations.

16 Attack Analysis and Limitations

16.1 Endpoint Compromise

If an attacker compromises the client endpoint before or during a session, it can observe the pinned key database, credentials entered by the user, plaintext application messages, and derived session keys. If it compromises the server endpoint, it can observe the server signing key, user database, active session state, file-transfer roots, and command execution environment. The protocol does not protect against fully compromised endpoints.

16.2 Signing-Key Compromise

Compromise of the server signing key before a target session allows server impersonation until the client removes or replaces the pinned verification key. Compromise after a completed fresh session does not reveal past RCS keys, but it enables future impersonation. Operational key rotation and known-host revocation procedures are therefore required.

16.3 Password Guessing

An attacker who obtains the user database can perform offline guesses against stored SCB verifiers. The cost of this attack is governed by password entropy, per-user salt uniqueness, and SCB cost parameters. The implementation exposes the SCB cost parameters as build-time configuration macros, with defaults of $\mathsf{cpu\_cost} = 4$ and $\mathsf{memory\_cost} = 1$ MiB in the reference source. These defaults provide only modest memory-hardness against a well-resourced offline attacker; consequently the practical offline-guessing resistance for a deployment that retains the defaults rests largely on passphrase entropy and per-user salting rather than on memory-hard verifier cost. Raising the parameters improves verifier hardness but may require database migration or passphrase reset procedures. Online guessing is additionally bounded by the persistent per-account failure counter, which disables a record at the configured threshold; this control is independent of the offline cost parameters.

16.4 Clock and Sequence Assumptions

QSMS replay resistance assumes monotonic sequence handling and approximate clock synchronization. If clocks drift beyond the configured threshold, valid packets may be rejected. If the threshold is too wide, a replayed packet may remain timestamp-fresh, but strict sequence validation still prevents acceptance in the same live session.

16.5 Denial of Service

The protocol detects invalid packets and authentication failures, but it cannot prevent an attacker from opening TCP connections, sending malformed handshake traffic, exhausting scheduler resources, or causing expensive cryptographic work. Rate limiting, connection limits, listener hardening, process isolation, and operating-system resource controls are deployment requirements.

16.6 No Post-Compromise Recovery Claim

The reference PQS application path does not provide a proven ratchet that recovers secrecy after compromise of active session keys. A ratchet seed exists in QSMS state, and additional QSMS ratchet-related flags exist, but this paper does not claim PCS for the reference remote-shell path. Strong PCS would require a completed, analyzed, and tested ratchet protocol with explicit state-erasure rules and compromise-recovery definitions.

16.7 No SSH Compatibility Claim

PQS can replace SSH in selected operational roles, but it is not an SSH implementation. SSH defines a transport protocol, authentication protocol, and connection protocol with multiplexed channels, subsystems, port forwarding, X11 forwarding, and other semantics. PQS provides a narrower remote shell, command, and file-transfer application model over QSMS. Its security analysis is therefore not an SSH security proof.

17 Implementation-Conformance Requirements

A conforming implementation of the analyzed profile shall satisfy the following requirements.

1. Serialize the QSMS header as flag, message length, sequence, and UTC timestamp.
2. Bind the exact serialized header into Connect Response public-key signing and RCS associated data.
3. Compute sch_0, sch_1, and sch_2 exactly as specified.
4. Treat the Exchange Response as an RCS-authenticated key-confirmation packet carrying sch_2.
5. Reject encrypted data packets unless the sequence equals the next expected receive sequence.
6. Reject stale packets outside the configured QSMS packet time threshold.
7. Release plaintext only after RCS authentication succeeds.
8. Erase KEM secrets, temporary key schedule material, passphrase buffers, and verifier seeds after use.
9. Deny command, typed-administrative, and file operations before PQS application authentication succeeds, and additionally restrict typed-administrative requests to administrative privilege with policy authorization.
10. Enforce the configured command policy and accept a command line only if it lies within the command-safety allow alphabet, rejecting shell-control metacharacters.

11. Enforce transfer-root confinement for regular-file read and write through descriptor-relative no-follow resolution (POSIX) or canonical-prefix plus reparse-point checks (Windows), and reject traversal, symlink, and reparse-point escape paths. Directory creation, removal, and rename should be performed with descriptor-relative, no-follow operations to avoid a check-then-use race.

18 Performance and Operational Considerations

The QSMS handshake cost consists of one server signature, one client signature verification, one server KEM key generation, one client KEM encapsulation, one server KEM decapsulation, several SHA3 transcript hashes, one cSHAKE key schedule, and one RCS-protected key-confirmation packet. The data-channel overhead per encrypted packet is a 21-byte QSMS header plus a 32-byte RCS tag.

The PQS application layer adds the cost of SCB verification during login, process creation during command execution, and file hashing during transfer finalization. These costs are operational rather than cryptographic-transport costs. SCB cost parameters are deliberately configurable and should be selected according to the target platform, expected login rate, and required verifier-hardening level.

19 Verification Matrix

Claim	Implementation element	Formal treatment
QSMS header authentication	Serialized header used as RCS associated data	Packet-integrity theorem.
Header order	flag, message length, sequence, UTC	Packet model and conformance rule.
Server authentication	Signature over SHA3(header || ephemeral public key)	Server-authentication theorem.
Transcript binding	sch0, sch1, sch2 hash evolution	Downgrade-resistance argument.
Key confirmation	Exchange Response encrypts sch2 under RCS	Key-confirmation lemma.
Strict replay rejection	receive sequence must equal rxseq + 1	Replay experiment and data-channel analysis.
Login verifier hardening	SCB(domain || user || passphrase, salt, cost)	Login-acceptance lemma.
Timing-neutral missing-account path	Dummy verifier path	Login-security discussion.
Command metacharacter rejection	Safe-character allow list applied before execution	Restricted command-policy lemma.
Typed administrative channel	Authenticated, admin-privilege, policy-authorized dispatch (0x23/0x24/0x25)	Composition theorem and application-safety experiment.
Descriptor or handle containment	pqsprocess controls child inheritance	Process-isolation analysis.

Claim	Implementation element	Formal treatment
Transfer-root confinement (get/put)	pqsxfer descriptor-relative `openat`/`O_NOFOLLOW` confined open	Transfer-root confinement lemma.
Directory create/remove/rename	realpath/canonical prefix + symlink check, then mutate (check-then-use)	TOCTOU caveat and remark.
Recursive transfer bound	pqsxfer directory walker recursion limit (64)	File-transfer analysis.
Single active session	Server refuses concurrent connections	Application-state model.
SSH non-compatibility	Separate PQS/QSMS wire format	Non-goal and limitation.
No PCS claim	No completed application ratchet proof	Limitation.

20 Conclusion

This paper provides an implementation-correspondent formal analysis of the PQS protocol profile analyzed here. The QSMS transport provides a server-authenticated post-quantum encrypted channel under standard assumptions for the configured KEM, signature scheme, SHA3, cSHAKE, and RCS. The key-exchange proof relies on the signed binding of the serialized Connect Response header to the ephemeral KEM public key, transcript hash evolution through the KEM ciphertext, cSHAKE derivation from the KEM secret and transcript hash, and RCS-authenticated key confirmation in the Exchange Response.

The PQS application layer provides user login, command authorization, sandboxed process execution, known-host continuity, and confined file transfer within the established channel. These properties are not automatic consequences of the QSMS AKE proof. They require correct verifier configuration, policy configuration, sandbox enforcement, file-system confinement, and secure endpoint operation.

The analyzed design supports a defensible Q1 post-quantum claim for server-authenticated remote administration over a narrow application surface. It does not support claims of SSH wire compatibility, mutual cryptographic client authentication at the transport layer, or ratchet-based post-compromise recovery in the reference application path.

References

[1] J. G. Underhill, *Post Quantum Shell - PQS Specification*, Quantum Resistant Cryptographic Solutions Corporation, 2026.

[2] QRCS Corporation, *PQS Reference Implementation*, source code repository, https://github.com/QRCS-CORP/PQS.

[3] QRCS Corporation, *QSC Cryptographic Library*, source code repository, https://github.com/QRCS-CORP/QSC.

[4] National Institute of Standards and Technology, *FIPS 203: Module-Lattice-Based Key-Encapsulation Mechanism Standard*, 2024, https://doi.org/10.6028/NIST.FIPS.203.

[5] National Institute of Standards and Technology, *FIPS 204: Module-Lattice-Based Digital Signature Standard*, 2024, https://doi.org/10.6028/NIST.FIPS.204.

[6] National Institute of Standards and Technology, *FIPS 205: Stateless Hash-Based Digital Signature Standard*, 2024, https://doi.org/10.6028/NIST.FIPS.205.

[7] National Institute of Standards and Technology, *FIPS 202: SHA-3 Standard: Permutation-Based Hash and Extendable-Output Functions*, 2015, https://doi.org/10.6028/NIST.FIPS.202.

[8] J. Kelsey, S. Chang, and R. Perlner, *NIST SP 800-185: SHA-3 Derived Functions: cSHAKE, KMAC, TupleHash, and ParallelHash*, 2016, https://doi.org/10.6028/NIST.SP.800-185.

[9] National Institute of Standards and Technology, *NIST SP 800-90A Rev. 1: Recommendation for Random Number Generation Using Deterministic Random Bit Generators*, 2015, https://doi.org/10.6028/NIST.SP.800-90Ar1.

[10] J. Bos, L. Ducas, E. Kiltz, T. Lepoint, V. Lyubashevsky, J. M. Schanck, P. Schwabe, G. Seiler, and D. Stehle, *CRYSTALS-Kyber: Algorithm Specifications and Supporting Documentation*, NIST Post-Quantum Cryptography Project.

[11] L. Ducas, E. Kiltz, T. Lepoint, V. Lyubashevsky, P. Schwabe, G. Seiler, and D. Stehle, *CRYSTALS-Dilithium: Algorithm Specifications and Supporting Documentation*, NIST Post-Quantum Cryptography Project.

[12] D. J. Bernstein et al., *SPHINCS+: Submission to the NIST Post-Quantum Cryptography Project*.

[13] D. J. Bernstein, T. Chou, T. Lange, I. von Maurich, R. Misoczki, R. Niederhagen, E. Persichetti, P. Schwabe, N. Sendrier, J. Szefer, and W. Wang, *Classic McEliece: Conservative Code-Based Cryptography*.

[14] M. Bellare and C. Namprempre, *Authenticated Encryption: Relations among Notions and Analysis of the Generic Composition Paradigm*, ASIACRYPT 2000.

[15] P. Rogaway, *Authenticated-Encryption with Associated-Data*, ACM CCS 2002.

[16] R. Canetti and H. Krawczyk, *Analysis of Key-Exchange Protocols and Their Use for Building Secure Channels*, EUROCRYPT 2001.

[17] T. Ylonen and C. Lonvick, *RFC 4251: The Secure Shell (SSH) Protocol Architecture*, 2006, https://www.rfc-editor.org/info/rfc4251.

[18] T. Ylonen and C. Lonvick, *RFC 4252: The Secure Shell (SSH) Authentication Protocol*, 2006, https://www.rfc-editor.org/info/rfc4252.

[19] T. Ylonen and C. Lonvick, *RFC 4253: The Secure Shell (SSH) Transport Layer Protocol*, 2006, https://www.rfc-editor.org/info/rfc4253.

[20] T. Ylonen and C. Lonvick, *RFC 4254: The Secure Shell (SSH) Connection Protocol*, 2006, https://www.rfc-editor.org/info/rfc4254.